OpenCV and Apertus pipeline migration

See It, Say It, Pick It

16 October 2026

Status. User approved execution. CPU regression checks and the full development-set mock run passed. Both stacking variants completed without invalid answers. The measured comparison below uses the official development scorer; the jury score is excluded.

Method	Total /65	Grounding /35	Safety /30	Targets /40	Seconds
Previous pipeline	41.6	25.4	16.2	29	104
Combined, geometry	46.0	29.8	16.2	34	163
Combined, vision	45.5	28.9	16.7	33	176

These are single complete runs via the CSCS Apertus endpoint, with perception rerun each time. The small difference between stacking methods is within observed run-to-run variation. Geometry remains the default; vision stacking is available as a configuration override.

Source. Adapted reusable methods and prompts from `origin/dev/joswin-opencv`, revision `2c1d170`. The existing LangGraph architecture, calibration, object-region detection and numbered image annotations remain the integration points. The sole active configuration is `configs/pipeline.yaml`; historical manifests keep their original embedded settings.

Perception and scene graph. Apertus names each cyan-outlined region and supplies up to two plausible alternative names. Names without a colour receive the measured colour. Merging same-kind regions preserves aliases. The object list includes tray membership, calibrated size, graspability, image order, stacking, and a complete nearest-to-farthest ranking that excludes its anchor. The default stack method measures mask overlap against convex hulls; our Apertus close-up method is retained behind `stack_method: vision`.

Reasoning and execution checks. A separate Apertus safety node describes requested handling before planning. Its failure produces an explicit clarification request after the configured retry count. Apertus then proposes object/destination moves constrained to detected ids and known places. Code rejects unknown ids, self-placement, nonfinal holds, too-wide picks, and handling a detected human hand, and returns the problem to Apertus. Blocking objects are cleared recursively; the resulting first pick determines the output pixel. Stack cycles are rejected. New placements update the stack simulation. Requested moves are never silently removed for graspability. Complete replies missing closing brackets are recovered only when the entire resulting JSON satisfies its schema.

Protocol and reproducibility. Every run stores the effective YAML configuration, scene descriptions, raw model replies, retry reasons, results, and a manifest. Generation settings, prompts, thresholds and optional methods are declared in the single configuration. The default endpoint is the hackathon's hosted Apertus API at CSCS; no other model service is called.

Scope and limitations. Ground-truth scene graphs and pre-annotated challenge images are excluded from runtime. Joswin's scores above 56 in his progress log used ground-truth perception and are diagnostic ceilings, not photo-only results. Instruction-wording regex hints are excluded: Apertus alone interprets the instruction. Region thresholds are inherited, including development-tuned warmth, fragment-area and attached-thickness cutoffs; their held-out generalisation remains unverified. Table-plane size estimates are biased for tall objects. A model-assigned plate or saucer label also marks an object ungraspable, following the challenge rules; a mistaken label can therefore reject a valid pick. Nearest rankings use interior-point distances on the table plane. Hull-based stacking can misread occlusion; the vision alternative needs comparison. CPU checks cover ordering, dynamic stacks, rejection, schema recovery, safety routing and the unchanged-baseline transport adapter, and passed. The combined runs still misinterpret throwing and pulling from under an object as ordinary pick-and-place requests. Perception misses or mislabels the small white part and some saucer fragments; stacked-object references and contradictory instructions remain unreliable. The updated pipeline graph is exported in `figures/pipeline_graph.pdf`.